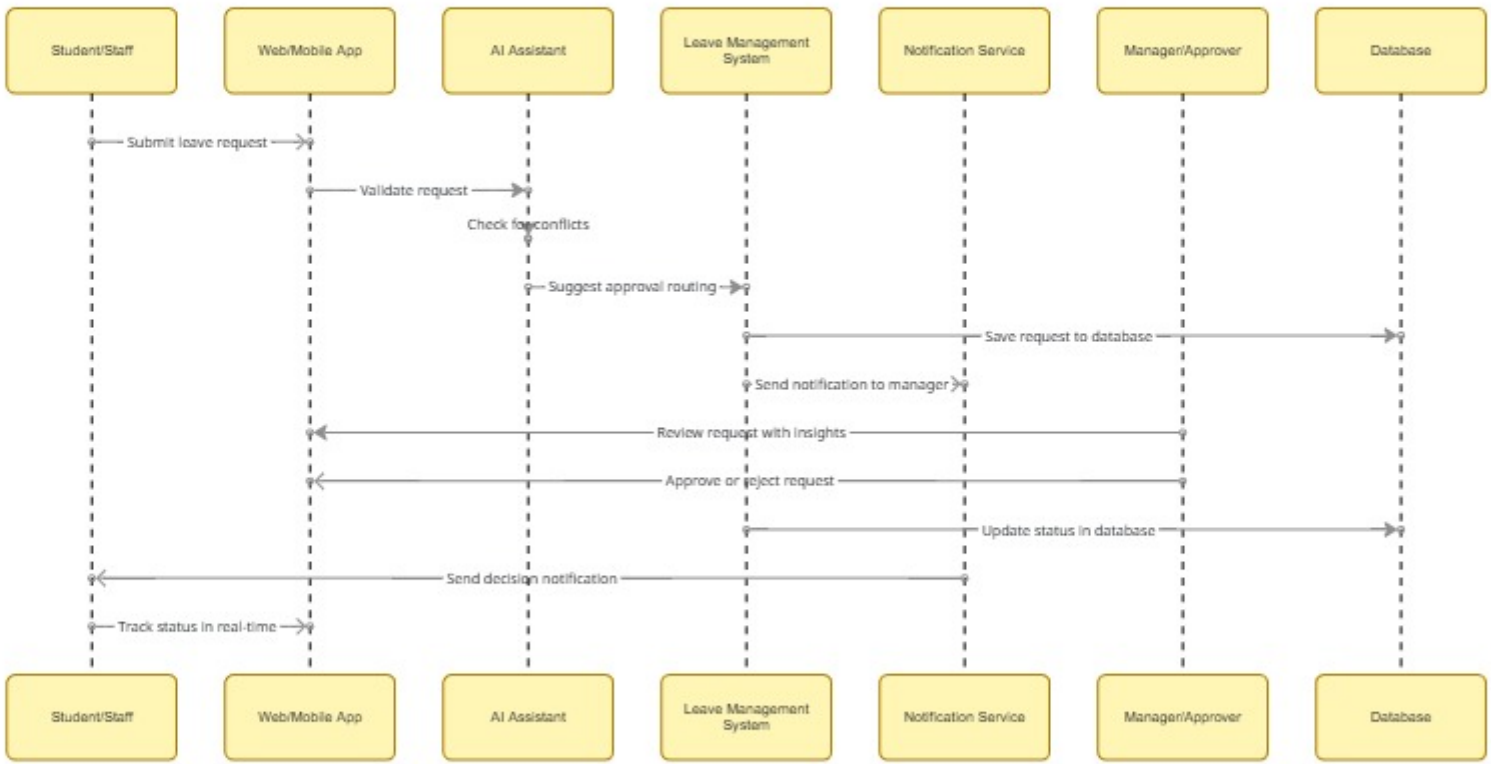
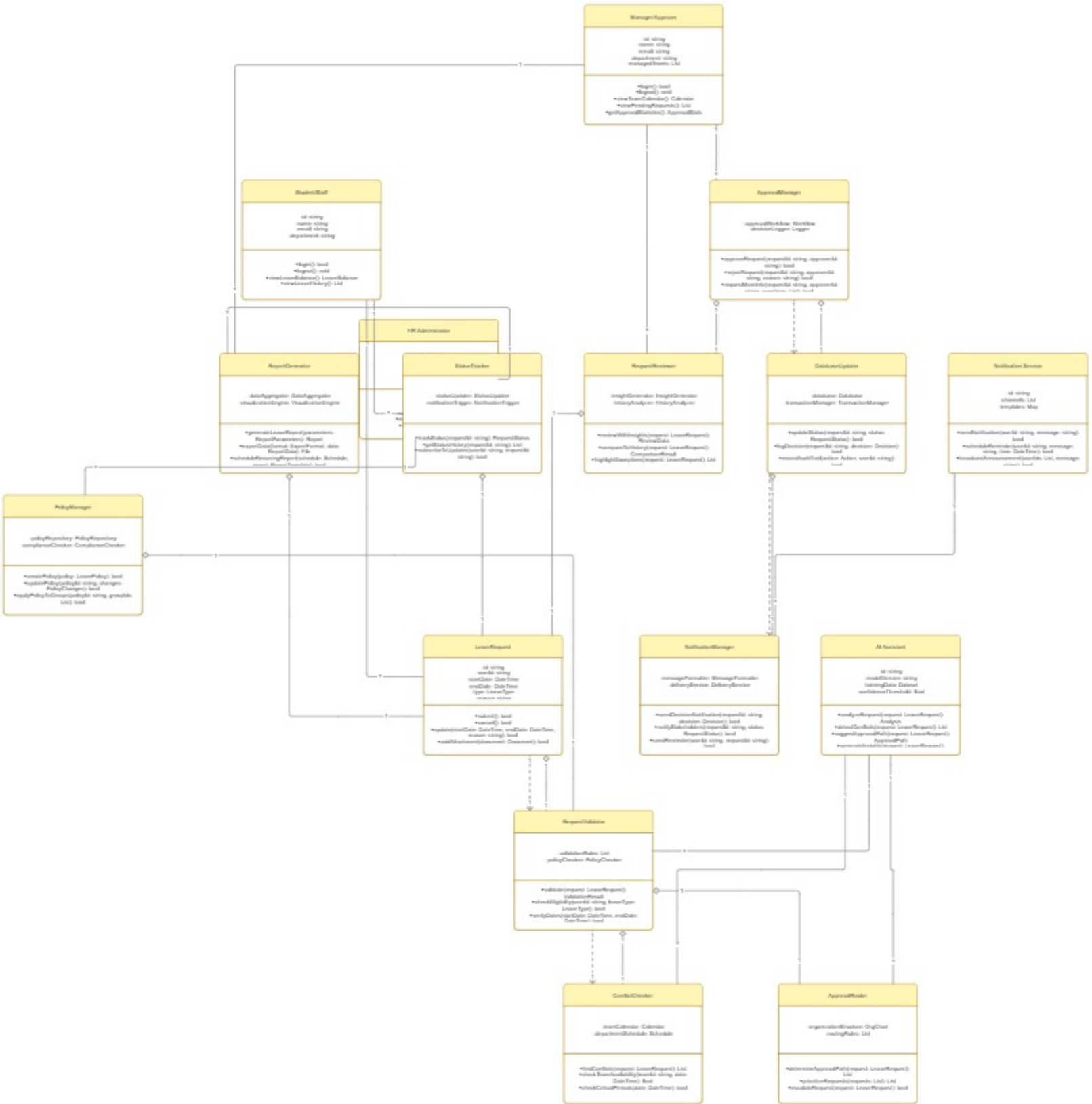
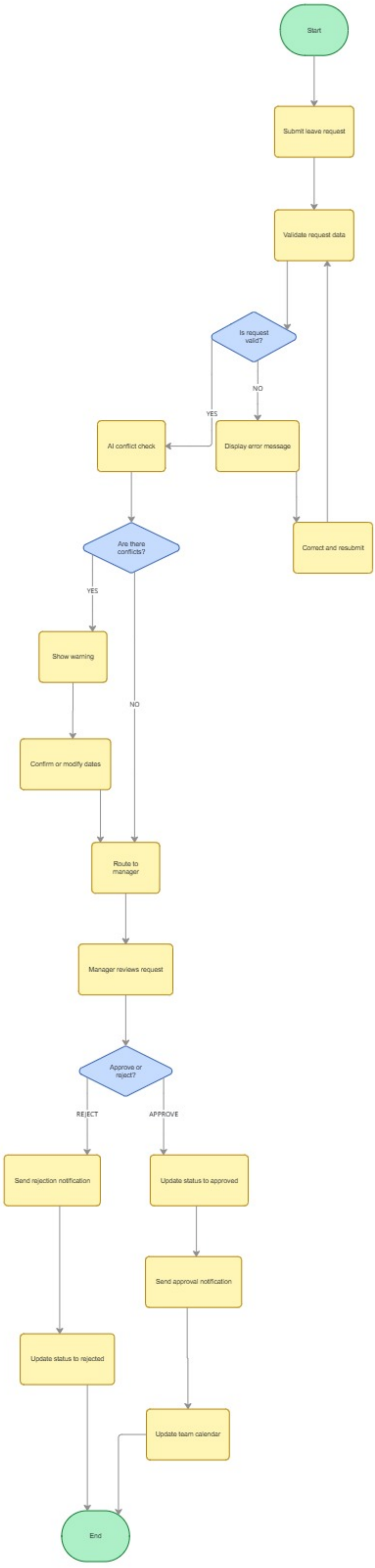
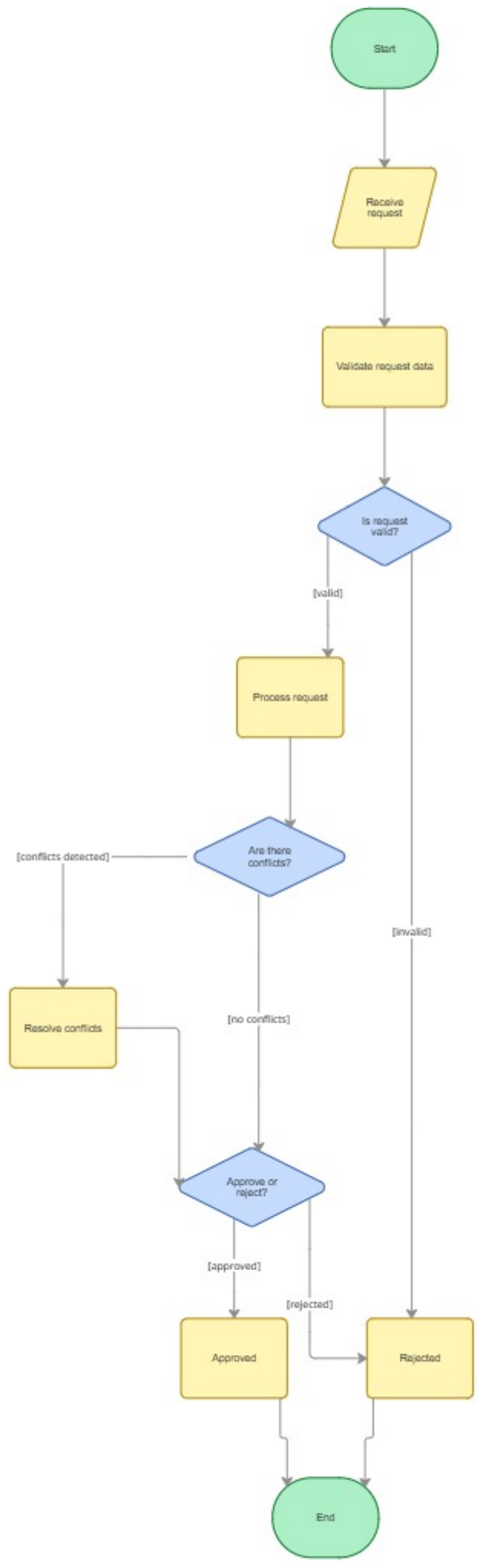
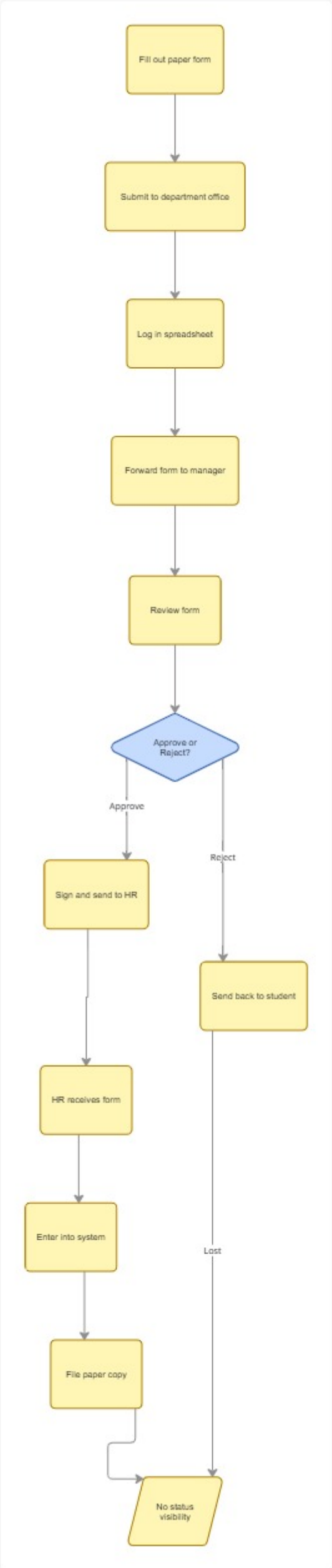
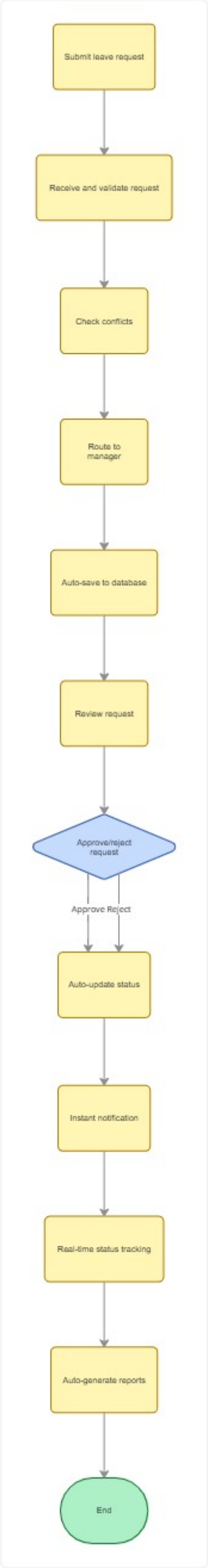

Student/Staff
Web/Mobile App
AI Assistant
Leave Management System
Notification Service
Manager/Approver
Database
Submit leave request
Validate request
Check for conflicts
Suggest approval routing
Save request to database
Send notification to manager
Review request with insights
Approve or reject request
Update status in database
Send decision notification
Track status in real-time
Student/Staff
Web/Mobile App
AI Assistant
Leave Management System
Notification Service
Manager/Approver
Database


ManagerApprover
id: string
name: string
email: string
department: string
managedTeams: List
+login(): bool
+logout(): void
+viewTeamCalendar(): Calendar
+viewPendingRequests(): List
+getApprovalStatistics(): ApprovalStats
StudentStaff
id: string
name: string
email: string
department: string
+login(): bool
+logout(): void
+viewLeaveBalance(): LeaveBalance
+viewLeaveHistory(): List
ApprovalManager
approvalWorkflow: Workflow
decisionLogger: Logger
+approveRequest(requestId: string, approverId: string): bool
+rejectRequest(requestId: string, approverId: string, reason: string): bool
+requestMoreInfo(requestId: string, approverId: string, questions: List): bool
HR Administrator
ReportGenerator
dataAggregator: DataAggregator
visualizationEngine: VisualizationEngine
+generateLeaveReport(parameters: ReportParameters): Report
+exportData(format: ExportFormat, data: ReportData): File
+scheduleRecurringReport(schedule: Schedule, recipients: List): bool
StatusTracker
statusUpdater: StatusUpdater
notificationTrigger: NotificationTrigger
+trackStatus(requestId: string): RequestStatus
+getStatusHistory(requestId: string): List
+subscribeToUpdates(userId: string, requestId: string): bool
RequestReviewer
insightGenerator: InsightGenerator
historyAnalyzer: HistoryAnalyzer
+reviewWithInsights(request: LeaveRequest): ReviewData
+compareToHistory(request: LeaveRequest): ComparisonResult
+highlightAnomalies(request: LeaveRequest): List
DatabaseUpdater
database: Database
transactionManager: TransactionManager
+updateStatus(requestId: string, status: RequestStatus): bool
+logDecision(requestId: string, decision: Decision): bool
+recordAuditTrail(action: Action, userId: string): bool
NotificationService
id: string
channels: List
templates: Map
+sendNotification(userId: string, message: string): bool
+scheduleReminder(userId: string, message: string, time: DateTime): bool
+broadcastAnnouncement(userIds: List, message: string): bool
PolicyManager
policyRepository: PolicyRepository
complianceChecker: ComplianceChecker
+createPolicy(policy: LeavePolicy): bool
+updatePolicy(policyId: string, changes: PolicyChanges): bool
+applyPolicyToGroups(policyId: string, groupIds: List): bool
LeaveRequest
id: string
userId: string
startDate: DateTime
endDate: DateTime
type: LeaveType
reason: string
+submit(): bool
+cancel(): bool
+update(startDate: DateTime, endDate: DateTime, reason: string): bool
+addAttachment(document: Document): bool
NotificationManager
messageFormatter: MessageFormatter
deliveryService: DeliveryService
+sendDecisionNotification(requestId: string, decision: Decision): bool
+notifyStakeholders(requestId: string, status: RequestStatus): bool
+sendReminder(userId: string, requestId: string): bool
AI Assistant
id: string
modelVersion: string
trainingData: Dataset
confidenceThreshold: float
+analyzeRequest(request: LeaveRequest): Analysis
+detectConflicts(request: LeaveRequest): List
+suggestApprovalPath(request: LeaveRequest): ApprovalPath
+approveSelfAssisted(request: LeaveRequest)
RequestValidator
validationRules: List
policyChecker: PolicyChecker
+validate(request: LeaveRequest): ValidationResult
+checkEligibility(userId: string, leaveType: LeaveType): bool
+verifyDates(startDate: DateTime, endDate: DateTime): bool
ConflictChecker
teamCalendar: Calendar
departmentSchedule: Schedule
+findConflicts(request: LeaveRequest): List
+checkTeamAvailability(teamId: string, date: DateTime): float
+checkCriticalPeriods(date: DateTime): bool
ApprovalRouter
organizationStructure: OrgChart
routingRules: List
+determineApprovalPath(request: LeaveRequest): List
+prioritizeRequests(requests: List): List
+escalateRequest(request: LeaveRequest): bool


Start
Submit leave request
Validate request data
Is request valid?
NO
YES
AI conflict check
Display error message
Correct and resubmit
Are there conflicts?
YES
NO
Show warning
Confirm or modify dates
Route to manager
Manager reviews request
Approve or reject?
REJECT
APPROVE
Send rejection notification
Update status to approved
Send approval notification
Update status to rejected
Update team calendar
End


Start
Receive request
Validate request data
Is request valid?
[valid]
[invalid]
Process request
Are there conflicts?
[conflicts detected]
[no conflicts]
Resolve conflicts
Approve or reject?
[approved]
[rejected]
Approved
Rejected
End


Fill out paper form
Submit to department office
Log in spreadsheet
Forward form to manager
Review form
Approve or Reject?
Approve
Reject
Sign and send to HR
Send back to student
HR receives form
Enter into system
Lost
File paper copy
No status visibility


Submit leave request
Receive and validate request
Check conflicts
Route to manager
Auto-save to database
Review request
Approve/reject request
Approve Reject
Auto-update status
Instant notification
Real-time status tracking
Auto-generate reports
End

Manual data entry - prone to errors

Physical handoff delays if office closed

Duplicate data entry - inefficient

Possibly physical reading - can get lost

No control or history - unreformed decisions

Manager unavailable? Process stalls.

Rework loop - student must restart

Another handoff - more delay

Third time entering same data

Physical storage - hard to retrieve

Student has no idea where their request is